# EVENT-DRIVEN NOTIFICATION ENGINE

Project Report & Resubmission Guide

Prepared for project work-experience resubmission

Technology: Node.js • Express.js • REST API • JSON • Automated tests

## Important scope note

This report describes a learning prototype. Event records are kept in memory, and notification processing is simulated. It does not currently send real email/SMS messages or use a persistent database or message broker.

## 1. Executive Summary

The Event-Driven Notification Engine demonstrates a consistent API for accepting notification events, validating their data, tracking their status, and simulating processing. It is intended to illustrate the basic event lifecycle and API error handling.

## 2. Problem Statement

Applications need to react to business events—such as an order being placed—by notifying a recipient. A small notification service can standardize how event data is accepted and how processing status is tracked.

## 3. Objectives

• Accept notification event payloads over HTTP.
• Validate required fields before accepting an event.
• Assign each event a unique ID and initial status.
• Allow event listing and retrieval.
• Simulate event processing and prevent duplicate processing.
• Return consistent JSON errors and verify behavior with automated tests.

## 4. Architecture

Client / API tester → Express REST API → request validation → in-memory event collection → simulated processor → status response.

## 5. API Endpoints

| Method & path | Purpose | Expected result |
|---|---|---|
| GET / | Service overview | 200 OK |
| GET /health | Health check | 200 OK; status UP |
| POST /events | Create event | 201 Created; status RECEIVED |
| GET /events | List events | 200 OK; event count and list |
| GET /events/:id | Retrieve event | 200 OK or 404 Not Found |
| POST /events/:id/process | Simulate processing | 200 OK; SENT, or 409 if repeated |

## 6. Example Event Payload

```
POST /events\nContent-Type: application/json\n\n{\n  "eventType": "OrderPlaced",\n  "recipient": "customer@exa
```

## 7. Validation and Error Handling

Missing or blank required fields return HTTP 400. Unknown event IDs and routes return HTTP 404. Reprocessing an event already marked SENT returns HTTP 409. Malformed JSON returns HTTP 400. Automated tests cover health checks, validation, event lifecycle, duplicate processing, and unknown routes.

## 8. How to Run

```
npm install\nnpm test\nnpm start
```

The service listens on http://localhost:3000 by default. Visit /health to verify the service is responding.

## 9. Test Evidence to Capture

Capture screenshots from the actual run showing: (1) `npm test` passing, (2) `/health` response, (3) successful event creation, (4) event processing and status SENT, and (5) invalid payload returning HTTP 400. Do not use fabricated screenshots or report tests as passing until they have been run.

## 10. Limitations

The prototype stores events in memory, so records are lost on restart. Processing only changes the status; it does not deliver a real notification. There is no database, message broker, authentication, retry policy, or production monitoring. These limitations must be disclosed in the submission.

## 11. Future Enhancements

• PostgreSQL persistence and database migrations.
• RabbitMQ or another broker for asynchronous processing.
• Retry with backoff and dead-letter handling.
• Real provider integration for email/SMS with secrets in environment variables.
• Authentication, rate limiting, structured logs, metrics, and CI pipeline.
• Docker packaging and deployment documentation.

## 12. Resubmission Checklist

■ Compare implementation with the original problem statement.
■ Install dependencies and run all tests.
■ Capture genuine test screenshots.
■ Confirm README setup steps work from a fresh terminal.
■ Remove credentials, `.env`, and `node_modules` from the ZIP.
■ Add the actual GitHub URL and follow the portal's file-format instructions.
■ Keep a backup of the earlier submission.

Prepared as a practical project report. Adapt it to the exact requirements in Zetheta's original problem statement and evaluation rubric.